LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

08

Linux Boot Process and System Initialization

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 Boot Process Overview
02 BIOS and UEFI
03 GRUB Bootloader
04 The Linux Kernel
05 Initramfs
06 systemd and PID 1
07 systemd Targets
08 Startup Services
09 Shutdown and Reboot
10 Boot Logs and Analysis
11 Kernel Parameters
12 Boot Troubleshooting
13 Interview Q&A
14 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

CHAPTER OBJECTIVE

Understand how a Linux system starts from power-on to a fully operational state. Learn the roles of BIOS/UEFI, GRUB, the Linux kernel, initramfs and systemd. Understand boot targets, system startup services, boot logs and common troubleshooting techniques used when Linux servers fail to boot.

1 What Happens When a Linux Server Starts?

When you power on a Linux server or start an EC2 instance, several components work together to initialize the operating system. The Linux boot process can be divided into these major stages:

1. BIOS or UEFI initialization.
2. Bootloader (GRUB).
3. Linux kernel loading.
4. Initramfs initialization.
5. Starting the init system (systemd).
6. Starting system services.
7. Reaching the configured boot target.

Linux Boot Process Overview

```
Power ON
   |
   v
BIOS / UEFI
   |
   v
GRUB Bootloader
   |
   v
Linux Kernel
   |
   v
Initramfs
   |
   v
systemd (PID 1)
   |
   v
System Services
   |
   v
Login / Multi-user Target
   |
   v
System Ready
```

Understanding this sequence is important because boot failures can happen at different stages.

2 BIOS and UEFI

BIOS and UEFI are firmware interfaces that initialize hardware and begin the operating system boot process.

The first thing the firmware does is run **POST** (Power-On Self-Test), which checks that the CPU, memory and basic hardware are working before anything else starts.

BIOS (Basic Input/Output System)

BIOS is the older firmware interface. Its responsibilities include:

- Initializing basic hardware.
- Performing hardware checks.
- Identifying bootable devices.

- Starting the bootloader.

BIOS reads the first sector of the boot disk (the MBR, 512 bytes), which contains the first stage of the bootloader.

UEFI (Unified Extensible Firmware Interface)

UEFI is the modern replacement for traditional BIOS. It provides:

- Support for modern disk partitioning using GPT.
- An EFI System Partition (ESP).
- Support for Secure Boot.
- More flexible boot management.

UEFI loads the bootloader as a `.efi` file from the EFI System Partition, usually mounted at `/boot/efi`.

BIOS vs UEFI

BIOS	UEFI
Older firmware interface	Modern firmware interface
Commonly uses MBR booting	Commonly uses GPT and EFI System Partition
Bootloader in the first sector of the disk	Bootloader is a `.efi` file in `/boot/efi`
No Secure Boot	Supports Secure Boot
More limited boot features	More advanced boot features
Legacy boot mode	Modern boot mode

Check Which Mode the System Booted In

```
# The /sys/firmware/efi directory exists only on UEFI systems
[ -d /sys/firmware/efi ] && echo "UEFI" || echo "BIOS"

# Check whether Secure Boot is enabled
mokutil --sb-state
```

> **DEVOPS NOTE**
>
> On cloud virtual machines, firmware and bootloader details are often managed by the cloud platform or image configuration. However, understanding the boot sequence helps when troubleshooting Linux startup failures.

3 Understanding GRUB Bootloader

GRUB (GRand Unified Bootloader) is a commonly used Linux bootloader. After firmware initialization, GRUB loads the selected Linux kernel and its initial RAM filesystem.

GRUB can also provide a boot menu where administrators can:

- Select an operating system.
- Choose a kernel version.
- Boot into recovery mode.
- Modify kernel boot parameters.

GRUB Configuration Locations

```
/boot/grub/grub.cfg
/boot/grub2/grub.cfg
/etc/default/grub
/etc/grub.d/
```

The exact location depends on the Linux distribution and firmware configuration.

Important Settings in /etc/default/grub

Setting	Purpose
`GRUB_TIMEOUT`	Seconds the boot menu waits before booting the default entry
`GRUB_DEFAULT`	Which menu entry or kernel boots by default
`GRUB_CMDLINE_LINUX`	Kernel parameters added to every boot

```
# View installed kernel versions
ls /boot/vmlinuz*

# Check the current kernel version
uname -r

# Show the kernel parameters used for the current boot
cat /proc/cmdline
```

Update GRUB Configuration

```
# Ubuntu and Debian
sudo update-grub

# Many RHEL-based distributions
sudo grub2-mkconfig -o /boot/grub2/grub.cfg
```

On RHEL-based systems, the correct output path depends on the system's boot configuration.

> **IMPORTANT**
> Do not manually edit the generated `grub.cfg` file. Modify the appropriate source configuration and regenerate it. Incorrect bootloader changes can make a server unbootable.

Editing Kernel Parameters at Boot (Temporary)

1. At the GRUB menu, highlight the boot entry and press `e`.
2. Find the line that starts with `linux`.
3. Add or change parameters at the end of that line.
4. Press `Ctrl + X` (or `F10`) to boot.

This change applies to the current boot only. It is commonly used for recovery, such as booting into rescue mode or resetting a root password.

4 Understanding the Linux Kernel

The Linux kernel is the core component of the operating system. It manages communication between hardware and software. Its responsibilities include:

- Process scheduling.
- Memory management.
- Device management.

- Filesystem operations.
- Networking.
- Security and access control.

During boot, GRUB loads the kernel into memory and transfers control to it.

```
# Check kernel version
uname -r
```

```
6.8.0-40-generic
```

```
# Display detailed kernel information
uname -a

# View loaded kernel modules
lsmod

# Show information about a module
modinfo ext4

# Load or remove a module
sudo modprobe <module>
sudo modprobe -r <module>

# Display hardware and kernel messages with readable timestamps
sudo dmesg -T
```

What Is Inside /boot?

File	Purpose
`vmlinuz-<version>`	Compressed Linux kernel image
`initrd.img-<version>` (Ubuntu) / `initramfs-<version>.img` (RHEL)	Initramfs image
`System.map-<version>`	Kernel symbol table, used for debugging
`config-<version>`	Kernel build configuration
`grub/` or `grub2/`	Bootloader files

The kernel initializes hardware, detects devices and prepares the system to start the first userspace process.

5 Understanding Initramfs

Initramfs means **Initial RAM Filesystem**. It is a temporary filesystem loaded into memory during the early boot process. Its purpose is to provide the tools and drivers needed to prepare the real root filesystem.

For example, the root filesystem may be stored on:

- LVM logical volumes.
- Encrypted disks.
- RAID devices.
- Network storage.

The system may need special drivers or tools to access these devices. Initramfs provides those components before the actual root filesystem is mounted.

Why Is Initramfs Important?

Suppose your Linux root filesystem is stored on an LVM logical volume. The kernel may need LVM tools and device drivers to discover and mount that volume. Initramfs provides the environment needed to perform these early boot operations.

Common Initramfs Troubleshooting Symptoms

- Kernel cannot find the root filesystem.
- System enters an emergency shell.
- Required storage drivers are missing.
- Filesystem or device discovery fails.

```
# Regenerate initramfs on Ubuntu/Debian
sudo update-initramfs -u

# Regenerate initramfs on many RHEL-based systems
sudo dracut -f

# View initramfs contents (Ubuntu/Debian)
lsinitramfs /boot/initrd.img-$(uname -r) | head

# View initramfs contents (RHEL-based)
sudo lsinitrd | head
```

Regeneration commands should only be used when there is a clear reason to regenerate the image.

6 Understanding systemd – The First Userspace Process

After the kernel initializes the system and mounts the real root filesystem, it starts the first userspace process. On most modern Linux distributions, this process is systemd. Its PID is `1`.

```
ps -p 1 -o pid,comm,args
```

Systemd is responsible for:

- Starting system services.
- Managing service dependencies.
- Mounting filesystems.
- Managing system targets.
- Handling system startup and shutdown.
- Monitoring services.

What Is PID 1?

PID 1 is the first process started in userspace. It becomes the parent or ancestor of many other system processes and is responsible for adopting orphaned processes.

SysV init vs systemd

Feature	SysV init (legacy)	systemd (modern)
Service startup	Sequential, one by one	Parallel, faster boot
Configuration	Shell scripts in `/etc/init.d/`	Declarative unit files
Dependencies	Controlled by script ordering	Explicit dependencies (`Requires` , `After`)
Logging	Separate log files	Central journal (journald)
Supervision	Limited	Automatic restart, cgroup tracking
Commands	`service` , `chkconfig`	`systemctl`

INTERVIEW LINE

Why did systemd replace SysV init?

Mainly for faster parallel boot, proper dependency management, better service supervision with automatic restarts, and centralized logging.

7 Understanding systemd Targets

Systemd uses targets to group services and define system operating states. Targets are similar in purpose to traditional SysV runlevels.

Target	Purpose
`poweroff.target`	Shut down the system
`rescue.target`	Single-user rescue environment
`multi-user.target`	Multi-user command-line environment
`graphical.target`	Multi-user environment with graphical login
`reboot.target`	Restart the system
`emergency.target`	Minimal emergency environment

```
# Check the default target
systemctl get-default

# Set command-line mode as default
sudo systemctl set-default multi-user.target

# Set graphical mode as default
sudo systemctl set-default graphical.target

# Switch target immediately
sudo systemctl isolate multi-user.target
```

WARNING

Changing or isolating targets on a remote production server can stop services or terminate your remote graphical session. Understand the impact before executing these commands.

Rescue vs Emergency Target

Feature	rescue.target	emergency.target
Filesystems	All local filesystems mounted	Only root, mounted read-only
Services	Minimal basic services	Almost none
Use when	Fixing a service or configuration problem	Fixing fstab, filesystem or serious boot problems

Both usually require the root password. To boot directly into one, add `systemd.unit=rescue.target` (or `emergency.target`) to the `linux` line in the GRUB menu.

Traditional Runlevels and systemd Targets

Traditional runlevel	systemd target
0	poweroff.target
1	rescue.target
3	multi-user.target
5	graphical.target
6	reboot.target

Runlevels 2 and 4 also map to `multi-user.target`. Modern systemd-based Linux distributions use targets rather than traditional runlevels.

8 Understanding the Linux Startup Process

After systemd starts, it activates the required services and targets based on dependencies and configuration. For example, a typical server may start:

- Network services.
- SSH service.
- Logging services.
- Time synchronization.
- Monitoring agents.
- Application services.
- Database services.

```
# Check running services
systemctl list-units --type=service --state=running

# Check services configured to start automatically
systemctl list-unit-files --type=service --state=enabled

# Check a specific service
systemctl status nginx

# Enable a service at boot
sudo systemctl enable nginx

# Start it immediately and enable it at boot
sudo systemctl enable --now nginx
```

> **IMPORTANT**
>
> A service being enabled does not guarantee that it will start successfully. Always check its actual status and logs.

9 Understanding Linux Shutdown and Reboot

Linux provides commands to shut down and restart the system.

```
# Reboot immediately
sudo reboot
sudo systemctl reboot

# Shut down immediately
sudo shutdown now

# Schedule shutdown after 10 minutes
sudo shutdown +10

# Cancel scheduled shutdown
sudo shutdown -c

# Power off
sudo systemctl poweroff
```

When Did the Server Last Boot?

```
# Last boot time
uptime -s
who -b

# Reboot and shutdown history
last -x reboot | head
```

Does the Server Need a Reboot?

After kernel or core library updates, a reboot is needed to use the new versions.

```
# Ubuntu: this file exists when a reboot is required
ls /var/run/reboot-required

# RHEL-based (from the dnf-utils / yum-utils package)
sudo needs-restarting -r
```

Before rebooting a production server, verify:

- Running application processes.
- Active deployments.
- Database operations.
- Connected users.
- Maintenance window and availability requirements.

10 Understanding Boot Logs

When a Linux server fails to start correctly, boot logs help identify the problem.

```
# View current boot logs
journalctl -b

# List all recorded boots
journalctl --list-boots

# View logs from the previous boot
journalctl -b -1

# View kernel messages from the current boot
journalctl -k -b

# View recent system logs
journalctl -n 100

# View errors from the current boot
journalctl -p err -b

# Follow logs in real time
journalctl -f
```

NOTE

Logs from previous boots are available only if the journal is persistent. See Chapter 5 for how to enable persistent journal storage.

Boot Performance Analysis

```
# Check boot duration
systemd-analyze
```

```
Startup finished in 3.2s (kernel) + 8.5s (userspace) = 11.7s
```

```
# Identify slow-starting services
systemd-analyze blame

# Check service dependency and ordering information
systemd-analyze critical-chain
```

DEVOPS NOTE

If a server boots slowly, `systemd-analyze blame` and `systemd-analyze critical-chain` help identify services or dependencies contributing to startup delays.

11 Understanding Kernel Parameters

Kernel parameters control certain Linux kernel behaviors. There are two kinds:

Type	Set using	Check using
Boot-time parameters	GRUB (`GRUB_CMDLINE_LINUX`)	`cat /proc/cmdline`
Runtime parameters	`sysctl`	`sysctl -a`

```
# View all runtime kernel parameters
sysctl -a

# Check a specific parameter
sysctl vm.swappiness

# Change a parameter temporarily (lost after reboot)
sudo sysctl vm.swappiness=10
```

Each sysctl parameter maps to a file under `/proc/sys`. For example, `vm.swappiness` is `/proc/sys/vm/swappiness`.

Make a Parameter Persistent

Configuration is commonly stored under `/etc/sysctl.conf` or `/etc/sysctl.d/`.

```
vm.swappiness=10
```

```
# Apply configuration
sudo sysctl --system
```

Common sysctl Parameters for DevOps

Parameter	Purpose
`net.ipv4.ip_forward`	Allows packet forwarding; required by Docker and Kubernetes networking
`vm.swappiness`	Controls the tendency to use swap
`fs.file-max`	System-wide maximum number of open files
`net.core.somaxconn`	Maximum queued incoming connections per listening socket
`vm.max_map_count`	Maximum memory map areas per process; Elasticsearch requires a higher value

> **IMPORTANT**
>
> Kernel parameters affect system behavior. Do not change production values without understanding the workload and testing the result.

12 Common Linux Boot Troubleshooting Scenarios

Scenario 1 Linux server gets stuck during boot

Possible causes:

- Filesystem errors.
- Incorrect `/etc/fstab` entries.
- Missing storage devices.
- Failed system services.
- Kernel or initramfs problems.

Useful commands after gaining console or recovery access:

```
journalctl -b
systemctl --failed
lsblk -f
cat /etc/fstab
```

Look for failed mounts and services.

Scenario 2 Server enters emergency mode

A common cause is an invalid filesystem mount entry in `/etc/fstab`.

```
# If working from an emergency shell, make root writable first
mount -o remount,rw /

# Check fstab and device UUIDs
cat /etc/fstab
blkid

# Correct the invalid entry, then validate it
systemctl daemon-reload
mount -a
```

The exact recovery procedure depends on the boot environment and filesystem state. To prevent this, use the `nofail` option for non-critical disks (see Chapter 6).

Scenario 3 SSH service does not start after boot

```
# Ubuntu
systemctl status ssh

# RHEL-based
systemctl status sshd

# Check logs
journalctl -u ssh -b

# Validate SSH configuration
sudo sshd -t
```

A configuration syntax error can prevent SSH from starting.

Scenario 4 A service fails during system startup

```
# Identify failed services
systemctl --failed

# Inspect the failed service
systemctl status nginx

# View its boot logs
journalctl -u nginx -b
```

Investigate configuration errors, dependencies, permissions and port conflicts.

Scenario 5 New kernel causes boot problems

A kernel update can sometimes introduce driver or compatibility issues. GRUB may provide an option to boot using an older installed kernel.

```
# After booting successfully, confirm the running kernel
uname -r

# Review kernel-related logs
journalctl -k -b

# RHEL-based: list kernels and set the old one as default
sudo grubby --info=ALL | grep -E "^kernel|^index"
sudo grubby --set-default /boot/vmlinuz-<old-version>
```

On Ubuntu, set `GRUB_DEFAULT` in `/etc/default/grub` and run `sudo update-grub`.

Do not remove the older working kernel until the new kernel has been validated.

Scenario 6 Server rebooted unexpectedly

```
# When did it reboot?
last -x reboot | head
journalctl --list-boots

# What happened just before the reboot?
journalctl -b -1 -e

# Kernel errors from the previous boot
journalctl -k -b -1 | grep -iE "panic|error|mce|watchdog"
```

Common causes:

- Kernel panic.
- Hardware or power failure.
- A watchdog restarting a hung system.
- A manual reboot, or an automatic reboot after updates.
- A cloud provider maintenance event.

Note: The OOM killer kills processes, not the whole server. It causes a reboot only if `vm.panic_on_oom` is enabled.

Scenario 7 Reset a forgotten root password

This requires console access to the server.

RHEL-based systems:

```
# 1. At the GRUB menu, press e and add rd.break to the end of the linux line
# 2. Press Ctrl + X to boot into the initramfs shell

mount -o remount,rw /sysroot
chroot /sysroot
passwd root

# Required on SELinux systems so file labels are fixed on next boot
touch /.autorelabel

exit
exit
```

Ubuntu:

```
# 1. At the GRUB menu, press e
# 2. On the linux line, replace "ro quiet splash" with "rw init=/bin/bash"
# 3. Press Ctrl + X

passwd root
exec /sbin/init
```

Anyone with console access can do this, which is why physical and console access must be protected. A GRUB password adds another layer of protection.

Scenario 8 EC2 instance does not boot or becomes unreachable after a change

1. Check the instance status checks in the AWS Console.
2. View the boot output using **Get system log** or **Get instance screenshot**.
3. If supported, connect using the **EC2 Serial Console** and fix the problem directly.

If none of these work, use a rescue instance:

```
# 1. Stop the broken instance and detach its root volume
# 2. Attach the volume to a helper instance in the same Availability Zone

lsblk
sudo mkdir /mnt/rescue
sudo mount /dev/nvme1n1p1 /mnt/rescue

# XFS: if both volumes came from the same image, they share a UUID
sudo mount -o nouuid /dev/nvme1n1p1 /mnt/rescue

# 3. Fix the problem, for example /mnt/rescue/etc/fstab
# 4. Unmount, detach, reattach as the root volume and start the instance
sudo umount /mnt/rescue
```

13 Common Interview Questions

Q1. Explain the Linux boot process.

The firmware initializes hardware, GRUB loads the kernel and initramfs, the kernel initializes the system, the real root filesystem is mounted, systemd starts as PID 1, and system services are started according to the configured target.

Q2. What is BIOS?

BIOS is traditional firmware responsible for hardware initialization and starting the boot process.

Q3. What is UEFI?

UEFI is a modern firmware interface that supports features such as GPT booting, EFI System Partitions and Secure Boot.

Q4. What is GRUB?

GRUB is a bootloader that loads the selected Linux kernel and initramfs.

Q5. What is the Linux kernel?

The kernel is the core of Linux. It manages CPU scheduling, memory, hardware, filesystems and networking.

Q6. What is initramfs?

Initramfs is a temporary filesystem loaded into memory during early boot. It provides tools and drivers required to access and mount the real root filesystem.

Q7. What is systemd?

Systemd is a system and service manager used by many modern Linux distributions. It starts as PID 1 and manages services and system targets.

Q8. What is PID 1?

PID 1 is the first userspace process. On most modern Linux distributions, it is systemd.

Q9. What is the difference between multi-user.target and graphical.target?

`multi-user.target` provides a multi-user command-line environment. `graphical.target` includes the multi-user environment with graphical login support.

Q10. How do you check failed services?

```
systemctl --failed
```

Q11. How do you check the previous boot logs?

```
journalctl -b -1
```

Q12. How do you identify slow-starting services?

```
systemd-analyze blame
```

Q13. What is the purpose of /etc/fstab?

It defines filesystem mount configuration, including filesystems that should be mounted automatically during startup.

Q14. What happens if an incorrect entry is added to /etc/fstab?

The system may fail to mount the filesystem and can enter emergency mode or experience boot delays, depending on the entry and mount options.

Q15. How do you troubleshoot a Linux server that is not booting?

Use console or recovery access, identify the boot stage where the failure occurs, inspect boot and kernel logs, check filesystem mounts, verify GRUB and initramfs configuration, and investigate failed services.

Q16. What is the difference between rescue mode and emergency mode?

Rescue mode mounts all local filesystems and starts a few basic services. Emergency mode mounts only the root filesystem as read-only and starts almost nothing. Emergency mode is used for serious problems such as broken fstab entries or filesystem errors.

Q17. Why did systemd replace SysV init?

systemd starts services in parallel for faster boot, manages dependencies explicitly, supervises and restarts services automatically, and provides centralized logging through journald.

Q18. How do you reset a forgotten root password?

With console access, edit the GRUB entry. On RHEL-based systems, add `rd.break`, remount `/sysroot` as read-write, `chroot` into it, run `passwd` and create `/.autorelabel`. On Ubuntu, boot with `rw init=/bin/bash` and run `passwd`.

Q19. How do you check whether a system booted using UEFI or BIOS?

If the directory `/sys/firmware/efi` exists, the system booted using UEFI. Otherwise, it used legacy BIOS.

Q20. How do you find out when and why a server rebooted?

Use `last -x reboot` or `who -b` to find when it rebooted. Then check `journalctl -b -1 -e` and kernel logs from the previous boot for panics, hardware errors, watchdog events or a manual reboot.

Q21. An EC2 instance does not boot after a configuration change. How do you recover it?

Check status checks and the system log or screenshot, and use the EC2 Serial Console if available. Otherwise, stop the instance, attach its root volume to a helper instance, mount it (using `nouuid` for XFS if needed), fix the configuration, then reattach the volume and start the instance.

14 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ The complete Linux boot sequence, including POST.
- ✓ BIOS and UEFI, and how to check which one a system uses.
- ✓ GRUB bootloader and editing kernel parameters at boot.
- ✓ Linux kernel initialization and the files in `/boot`.
- ✓ Initramfs and root filesystem mounting.
- ✓ systemd, PID 1, and SysV init vs systemd.
- ✓ systemd targets, rescue vs emergency mode, and traditional runlevels.
- ✓ Automatic service startup.
- ✓ Reboot and shutdown commands, and reboot history.
- ✓ Boot and kernel logs using journalctl.
- ✓ Boot performance analysis using systemd-analyze.
- ✓ Kernel parameters and sysctl.
- ✓ Common boot failure troubleshooting, root password reset and EC2 boot recovery.

PRACTICAL EXERCISES

1. Check the current kernel version using `uname -r`.
2. Identify the process running with PID 1.
3. Check the default systemd target.
4. List currently running services.
5. Identify failed services using `systemctl --failed`.
6. View logs from the current boot using `journalctl -b`.
7. Check previous boot logs using `journalctl -b -1`.
8. Analyze boot duration using `systemd-analyze`.
9. Identify slow-starting services using `systemd-analyze blame`.
10. Inspect `/etc/fstab` and verify its entries using `mount -a` (do not modify production entries for practice).
11. Check whether your system booted using UEFI or BIOS.
12. View the kernel parameters of the current boot using `cat /proc/cmdline`.
13. Find the last reboot time using `who -b` and `last -x reboot`.

NEXT CHAPTER

Chapter 09: Linux Security and Hardening

You will learn how to secure Linux servers using SSH hardening, firewall configuration, security updates, SELinux, AppArmor, sudo policies, auditing and other production security practices.

END OF CHAPTER 8